

Popular Machine Learning Methods: Idea, Practice and Math

Part 2, Chapter 2, Section 1:
Linear Regression

Yuxiao Huang

Data Science, Columbian College of Arts & Sciences
George Washington University

Spring 2026

Reference

- This set of slides was largely built on the following 7 wonderful books and a wide range of fabulous papers:

HML Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition)

PML Python Machine Learning (3rd Edition)

ESL The Elements of Statistical Learning (2nd Edition)

PRML Pattern Recognition and Machine Learning

NND Neural Network Design (2nd Edition)

LFD Learning From Data

RL Reinforcement Learning: An Introduction (2nd Edition)

- For most materials covered in the slides, we will specify their corresponding books and papers for further reference.

Code Example

- See related code example in github repository:
[/p2_c2_s2_linear_regression/code_example](#)

Table of Contents

- 1 Learning Objectives
- 2 Linear Regression
- 3 Linear Regression in Sklearn
- 4 Training Linear Regression
- 5 Appendix
- 6 Bibliography

Learning Objectives: Expectation

- It is expected to understand
 - the idea of linear regression
 - the pros and cons of sklearn LinearRegression and SGDRegressor
 - the good practice for using sklearn LinearRegression and SGDRegressor
 - the idea of Loss function
 - the idea of Optimization
 - the idea of Taylor Series Expansion
 - the idea and implementation of the Normal Equation
 - the idea and implementation of Gradient Descent, including:
 - Batch Gradient Descent (BGD)
 - Stochastic Gradient Descent (SGD)
 - Mini-Batch Gradient Descent (MBGD)
 - the pros and cons of BGD / SGD / MBGD
 - the good practice for using BGD / SGD / MBGD

Learning Objectives: Recommendation

- It is recommended to understand
 - the math of Taylor Series Expansion
 - the math of the Normal Equation
 - the math of Gradient Descent, including:
 - Batch Gradient Descent (BGD)
 - Stochastic Gradient Descent (SGD)
 - Mini-Batch Gradient Descent (MBGD)
 - the analysis of time complexity of the normal equation and SGD

Kaggle Competition: Predicting House Price



Figure 1: Kaggle competition: predicting house price. Picture courtesy of Kaggle.

- House Prices (Advanced Regression Techniques) dataset:

- features: 79 explanatory variables describing (almost) every aspect of residential homes in Ames, Iowa
- target: the sale price of homes

House Prices Dataset

Table 1: The first 7 features and target (SalePrice) of House Prices dataset.

Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	SalePrice
1	60	RL	65.0	8450	Pave	NaN	208500
2	20	RL	80.0	9600	Pave	NaN	181500
3	60	RL	68.0	11250	Pave	NaN	223500
4	70	RL	60.0	9550	Pave	NaN	140000
5	60	RL	84.0	14260	Pave	NaN	250000

- The goal of this Kaggle competition is using the features (see the first 7 in table 1) to predict the target, SalePrice.
- Since SalePrice can take infinite number of values, it is a *Continuous* variable.
- We call this kind of prediction (where the target is a continuous variable) *Regression*.
- We will apply the simplest regression model (a.k.a., *Regressor*), *Linear Regression*, to this competition.

Mathematical Model

- A key reason for linear regression being the simplest regression model is that, it assumes a linear relationship between the features and target (hence the name).
- Concretely, in linear regression the target is modeled as a weighted sum of features:

$$\hat{y} = b + w_1 x_1 + \dots + w_n x_n. \quad (1)$$

Here:

- $\hat{y}$ is the $m \times 1$ predicted target vector (with m being the number of samples)
- $x_1, \dots, x_n$ are the $m \times 1$ feature vectors
- b is the bias
- $w_1, \dots, w_n$ are the weights of $x_1, \dots, x_n$ (with n being the number of features)
- We can also write eq. (1) in matrix form:

$$\hat{y} = 1b + Xw = \begin{bmatrix} 1 & X \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^T = \begin{bmatrix} 1 & X \end{bmatrix} \theta. \quad (2)$$

Here:

- $\mathbf{1}$ is a $m \times 1$ vector, full of 1s
- X is the $m \times n$ feature matrix:

$$X = \begin{bmatrix} x_1 & \dots & x_n \end{bmatrix} \quad (3)$$

- w is the $n \times 1$ weight vector:

$$w = \begin{bmatrix} w_1 & \dots & w_n \end{bmatrix}^T \quad (4)$$

- θ is the $(n+1) \times 1$ parameter vector:

$$\theta = \begin{bmatrix} b & w_1 & \dots & w_n \end{bmatrix}^T \quad (5)$$

Architecture

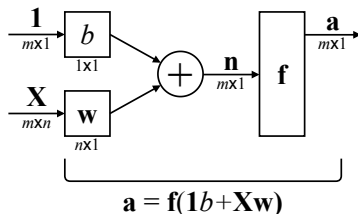


Figure 2: The architecture of linear regression.

- In the architecture of linear regression:

- $\mathbf{1}$ is a vector full of 1s and $\mathbf{X}$ a feature matrix
- b is the bias and $\mathbf{w}$ the weight vector
- $\mathbf{n}$ is the *Net Input* vector:

$$\mathbf{n} = \mathbf{1}b + \mathbf{X}\mathbf{w} \quad (6)$$

- $\mathbf{f}$ is the *Activation Function* vector (here it is the identity function vector):

$$\mathbf{f}(\mathbf{n}) = \mathbf{n} \quad (7)$$

- $\mathbf{a}$ is the output vector:

$$\mathbf{a} = \hat{\mathbf{y}} = \mathbf{f}(\mathbf{n}) = \mathbf{f}(\mathbf{1}b + \mathbf{X}\mathbf{w}) = \mathbf{1}b + \mathbf{X}\mathbf{w} \quad (8)$$

A 2D Example: Simple Linear Regression

Table 1: The first 7 features and target (SalePrice) of House Prices dataset.

Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	SalePrice
1	60	RL	65.0	8450	Pave	NaN	208500
2	20	RL	80.0	9600	Pave	NaN	181500
3	60	RL	68.0	11250	Pave	NaN	223500
4	70	RL	60.0	9550	Pave	NaN	140000
5	60	RL	84.0	14260	Pave	NaN	250000

- Suppose we want to use only one feature say, LotFrontage, to predict the target, SalePrice. Then linear regression composed of the two variables is

$$\widehat{\text{SalePrice}} = b + w \times \text{LotFrontage}. \quad (9)$$

Here:

- $\widehat{\text{SalePrice}}$ is the predicted target vector
- LotFrontage is the feature vector
- b is the bias
- w is the weight of LotFrontage
- Liner regression with only one feature is also called *Simple Linear Regression*.

Visualizing the 2D Data

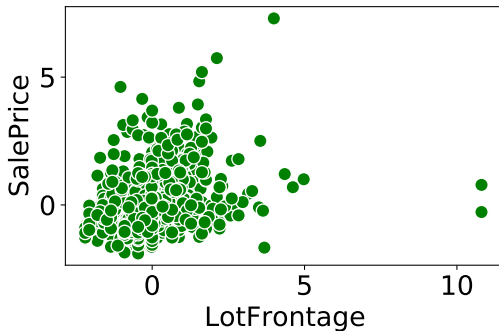


Figure 3: The scatter plot between feature LotFrontage and target SalePrice.

Visualizing the Simple Linear Regression

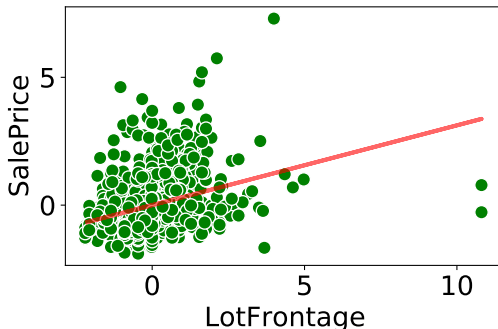


Figure 4: Simple linear regression with feature LotFrontage and target SalePrice.

- As shown in fig. 4, in this 2D case linear regression is a line.

A 3D Example: Multiple Linear Regression

Table 1: The first 7 features and target (SalePrice) of House Prices dataset.

Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	SalePrice
1	60	RL	65.0	8450	Pave	NaN	208500
2	20	RL	80.0	9600	Pave	NaN	181500
3	60	RL	68.0	11250	Pave	NaN	223500
4	70	RL	60.0	9550	Pave	NaN	140000
5	60	RL	84.0	14260	Pave	NaN	250000

- Suppose now we want to use two features say, LotFrontage and LotArea, to predict the target, SalePrice. Then linear regression composed of the three variables is

$$\widehat{\text{SalePrice}} = b + w_1 \times \text{LotFrontage} + w_2 \times \text{LotArea}. \quad (10)$$

Here:

- $\widehat{\text{SalePrice}}$ is the predicted target vector
- LotFrontage and LotArea are the feature vectors
- b is the bias
- w_1 and w_2 are the weight of LotFrontage and LotArea
- Liner regression with multiple features is also called *Multiple Linear Regression*.

Visualizing the 3D Data

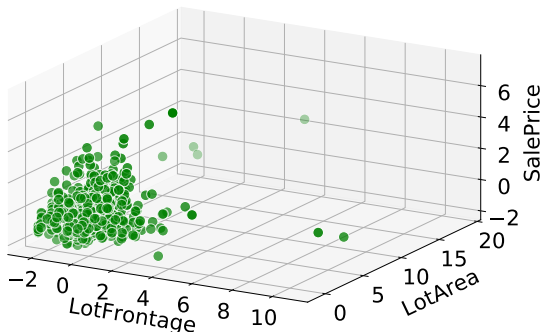


Figure 5: The scatter plot between feature LotFrontage, LotArea and target SalePrice.

Visualizing the Multiple Linear Regression

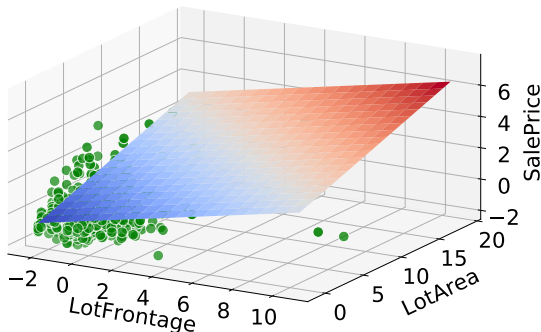


Figure 6: Multiple linear regression with feature LotFrontage, LotArea and target SalePrice.

- As shown in fig. 6, in this 3D case linear regression is a 2D plane.

Sklearn LinearRegression: Code Example

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - 1 cell 62

Sklearn SGDRegressor: Code Example

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - ① cell 63

LinearRegression VS SGDRegressor

- **Q:** Since there are two sklearn implementations for linear regression, which one shall we use?

LinearRegression VS SGDRegressor

- **Q:** Since there are two sklearn implementations for linear regression, which one shall we use?
- **A:** It is largely based on two factors:
 - the number of samples in the data, m
 - the number of features in the data, n



Good practice

- When m is very large (e.g., millions):
 - it is recommended to use SGDRegressor
 - When m is relatively small (e.g., thousands):
 - when $m \gg n$:
 - it is recommended to use LinearRegression (which is faster)
 - when $m \ll n$:
 - it is recommended to use SGDRegressor (which is faster)
- See the explanation of the good practice in Appendix (pages 64 to 66).

The Goal

- The goal of training linear regression is tweaking the parameters of the model in such a way that we can improve the model.
- There are two key elements in training linear regression:
 - Loss Function
 - Optimization
- The *Loss Function* of linear regression:
 - is a function of the parameters of the model
 - is a measurement of the *badness* of the model
- The *Optimization* for linear regression is a method that tweaks the parameters by minimizing the loss function (as it measures how bad the model is).

Mean Squared Error

- One popular loss function of linear regression is the *Mean Squared Error* (MSE), $\mathcal{L}(\boldsymbol{\theta})$, which measures the average squared difference between the real target value and the predicted target value, across all the samples:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (y^i - \hat{y}^i)^2 \quad \text{where} \quad \hat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i. \quad (11)$$

Here:

- m is the number of samples in the data
- $y^i / \hat{y}^i$ is the real / predicted target value of sample i
- $x_1^i, \dots, x_n^i$ are the feature values of sample i
- $\boldsymbol{\theta} = [b \ w_1 \dots w_n]^\top$ is the $(n+1) \times 1$ parameter vector
- We can decompose eq. (11) as

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \underbrace{(y^i - \hat{y}^i)^2}_{\substack{\text{error} \\ \text{squared error}}} \quad \underbrace{\hspace{1.5cm}}_{\substack{\text{sum of squared error} \\ \text{mean squared error}}} . \quad (12)$$

Surface and Contour Plot of MSE

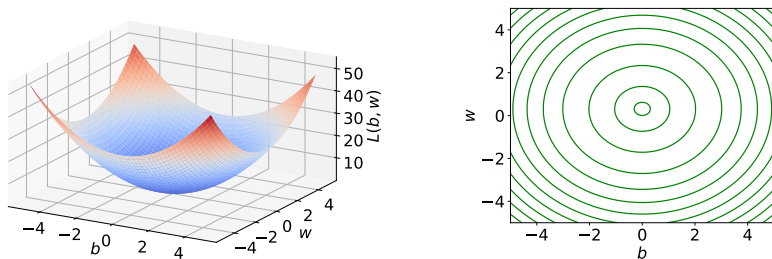


Figure 7: The surface and contour plot of training MSE of the linear regression in eq. (9).

- Fig. 7 shows the surface and contour plot of MSE of the linear regression in eq. (9):

$$\widehat{\text{SalePrice}} = b + w \times \text{LotFrontage}. \quad (9)$$

- As fig. 7 shows, both linear regression and MSE are function of parameters b and w .

Optimal Solution

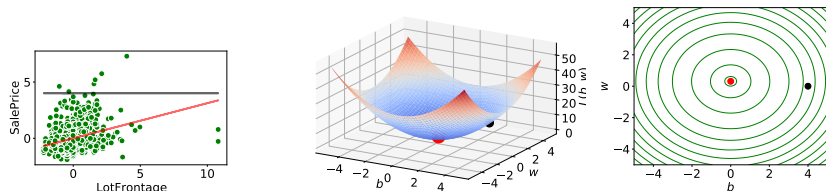


Figure 8: Two linear regression models (left) and their training MSE (middle and right).

- The black line in the left panel of fig. 8 and the black dot in the middle and right panel correspond to the following parameter setting:
 - $b = 4, w = 0$
- The red line in the left panel of fig. 8 and the red dot in the middle and right panel correspond to the following parameter setting:
 - $b = 0, w = 0.3$
- The parameter setting that leads to the lowest MSE are sometimes called the *Optimal Solution*.
- Based on the position of the red dot in the middle and right panel of fig. 8, in this case $b = 0$ and $w = 0.3$ are the optimal solution.

Optimization

- The goal of optimization is finding the optimal solution.
- When the loss (e.g., MSE) measures how bad the model is,
 - the lower the loss, the better the model
 - as a result, optimization entails *minimizing* the loss
- When the loss (e.g., Likelihood, see [/p2_c2_s3_logistic_regression](#)) measures how good the model is,
 - the higher the loss, the better the model
 - as a result, optimization entails *maximizing* the loss
- Interestingly, we may want to maximize the loss with respect to one part of the model and, at the same time, minimize the loss with respect to the other part of the model.
- One such model is the *Generative Adversarial Networks* (see [/p3_c3_s1_deep_generative_models](#)).

First-Order and Second-Order Optimization

- For linear regression, we will focus on two training methods, both belong to *First-Order Optimization*, which, as the name suggests, is based on the *first-order* derivative of the loss.
- See NND: Chap 9 for a nice discussion of a training method (*Newton's Method*) that belongs to *Second-Order Optimization*, which, as the name suggests, is based on the *second-order* derivative of the loss.
- It turns out that both first-order and second-order optimization estimate the optimal solution (that leads to the minimum loss) by exploring the relationship between the loss and optimal solution.
- To capture such relationship, we need to introduce a very important concept named *Taylor Series Expansion*.

Taylor Series Expansion

- For an analytic function (i.e., a function that is infinitely differentiable), $F(\theta)$, its Taylor series expansion about a point, θ^* , is

$$\begin{aligned}
 F(\theta) = & F(\theta^*) + \left. \frac{d}{d\theta} F(\theta) \right|_{\theta=\theta^*} (\theta - \theta^*) \\
 & + \frac{1}{2} \left. \frac{d^2}{d\theta^2} F(\theta) \right|_{\theta=\theta^*} (\theta - \theta^*)^2 \\
 & + \dots \\
 & + \frac{1}{n!} \left. \frac{d^n}{d\theta^n} F(\theta) \right|_{\theta=\theta^*} (\theta - \theta^*)^n \\
 & + \dots
 \end{aligned} \tag{13}$$

Here $n!$ is the factorial of n :

$$n! = \prod_{i=1}^n i, \tag{14}$$

and $\left. \frac{d^n}{d\theta^n} F(\theta) \right|_{\theta=\theta^*}$ the n th-order derivative of $F(\theta)$ evaluated at point θ^* .

- To have the equal sign in eq. (13), we need to include all the derivatives of $F(\theta)$ in the equation (this is why we require $F(\theta)$ to be infinitely differentiable).

Nth-Order Approximation

- If we do not include all the derivatives of $F(\theta)$, but instead, only include up to n th-order derivative in the equation, then the following Taylor series expansion is the *Nth-Order Approximation* of $F(\theta)$:

$$\begin{aligned}
 F(\theta) \approx & F(\theta^*) + \left. \frac{d}{d\theta} F(\theta) \right|_{\theta=\theta^*} (\theta - \theta^*) \\
 & + \frac{1}{2} \left. \frac{d^2}{d\theta^2} F(\theta) \right|_{\theta=\theta^*} (\theta - \theta^*)^2 \\
 & + \dots \\
 & + \frac{1}{n!} \left. \frac{d^n}{d\theta^n} F(\theta) \right|_{\theta=\theta^*} (\theta - \theta^*)^n.
 \end{aligned} \tag{15}$$

- In eq. (15), function $F(\theta)$ only has one parameter, θ .
- However, in reality $F(\boldsymbol{\theta})$ may have multiple parameters: $F(\boldsymbol{\theta}) = F([\theta_1 \cdots \theta_n]^\top)$.
- When this is the case, we need to write eq. (15) in matrix form:

$$\begin{aligned}
 F(\boldsymbol{\theta}) \approx & F(\boldsymbol{\theta}^*) + \nabla F(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\
 & + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\top \nabla^2 F(\boldsymbol{\theta}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\
 & + \dots
 \end{aligned} \tag{16}$$

Gradient

- In eq. (16)

$$\begin{aligned}
 F(\boldsymbol{\theta}) \approx & F(\boldsymbol{\theta}^*) + \nabla F(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\
 & + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\top \nabla^2 F(\boldsymbol{\theta}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\
 & + \dots
 \end{aligned} \tag{16}$$

$\nabla F(\boldsymbol{\theta})$ is the *Gradient* of $F(\boldsymbol{\theta})$:

$$\nabla F(\boldsymbol{\theta}) = \left[\frac{\partial}{\partial \theta_1} F(\boldsymbol{\theta}) \cdots \frac{\partial}{\partial \theta_n} F(\boldsymbol{\theta}) \right]^\top, \tag{17}$$

where the i th element, $\frac{\partial}{\partial \theta_i} F(\boldsymbol{\theta})$, is the first-order *partial* derivative of function $F(\boldsymbol{\theta})$ with respect to parameter θ_i .

- Eq. (17) shows that the gradient, $\nabla F(\boldsymbol{\theta})$, measures how fast the function, $F(\boldsymbol{\theta})$, changes with the parameters, $\boldsymbol{\theta}$:
 - if $\frac{\partial}{\partial \theta_i} F(\boldsymbol{\theta}) > 0$, the higher $\frac{\partial}{\partial \theta_i} F(\boldsymbol{\theta})$ the faster $F(\boldsymbol{\theta})$ increases when θ_i increases
 - if $\frac{\partial}{\partial \theta_i} F(\boldsymbol{\theta}) < 0$, the lower $\frac{\partial}{\partial \theta_i} F(\boldsymbol{\theta})$ the faster $F(\boldsymbol{\theta})$ decreases when θ_i increases
 - if $\frac{\partial}{\partial \theta_i} F(\boldsymbol{\theta}) = 0$, $F(\boldsymbol{\theta})$ does not change when θ_i changes

First-Order Approximation

- Based on eq. (16),

$$\begin{aligned}
 F(\boldsymbol{\theta}) \approx & F(\boldsymbol{\theta}^*) + \nabla F(\boldsymbol{\theta})^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\
 & + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\top \nabla^2 F(\boldsymbol{\theta}) \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \\
 & + \dots
 \end{aligned} \tag{16}$$

the *First-Order Approximation* (which only includes up to the first-order derivative) of $F(\boldsymbol{\theta})$ about a point $\boldsymbol{\theta}^*$ is

$$F(\boldsymbol{\theta}) \approx F(\boldsymbol{\theta}^*) + \nabla F(\boldsymbol{\theta})^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*). \tag{18}$$

- By replacing the function in eq. (18) ($F(\boldsymbol{\theta})$) with MSE ($\mathcal{L}(\boldsymbol{\theta})$), the first-order approximation of $\mathcal{L}(\boldsymbol{\theta})$ about the optimal solution (which minimizes $\mathcal{L}(\boldsymbol{\theta})$), $\boldsymbol{\theta}^*$, is

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^*) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*). \tag{19}$$

LinearRegression and SGDRegressor: Open the Blackbox

- Earlier we discussed two sklearn implementations of linear regression:
 - LinearRegression
 - SGDRegressor
- However, we basically treated them as a blackbox.
- Let us open the blackbox!
- It turns out that, the two tools use different methods for training:
 - LinearRegression uses a method named the *Normal Equation*
 - SGDRegressor uses a method named *Gradient Descent* (a.k.a., *Steepest Descent*)
- While the normal equation and gradient descent use different ways to find the optimal solution (which minimizes MSE), they do have something in common.
- Concretely, both of them:
 - belong to first-order optimization
 - are closely related to the first-order estimation of MSE (eq. (19)):

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^*) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) \quad (19)$$

- estimate the optimal solution by exploring its relationship with MSE

First-Order Condition

- The normal equation makes use of the following relationship between MSE ($\mathcal{L}(\boldsymbol{\theta})$) and the optimal solution ($\boldsymbol{\theta}^*$):

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0. \quad (20)$$

- Eq. (20) says that, the first-order derivative (i.e., gradient) of MSE ($\mathcal{L}(\boldsymbol{\theta})$) evaluated at the optimal solution ($\boldsymbol{\theta}^*$) is zero.
- This equation is also called the *First-Order Condition*.
- See the proof of first-order condition in Appendix (page 69).

The Normal Equation

- Eq. (20) shows the first-order condition:

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0. \quad (20)$$

- It turns out that we can solve the first-order condition analytically to obtain the optimal solution, $\boldsymbol{\theta}^*$:

$$\boldsymbol{\theta}^* = ([\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}])^{-1} [\mathbf{1} \quad \mathbf{X}]^\top \mathbf{y}. \quad (21)$$

Here:

- $\boldsymbol{\theta}^*$ is the $(n+1) \times 1$ optimal solution vector (with n being the number of features):

$$\boldsymbol{\theta} = [b^* \quad w_1^* \dots w_n^*]^\top \quad (22)$$

- $\mathbf{1}$ is a $m \times 1$ vector, full of 1s (with m being the number of samples)
- $\mathbf{X}$ is the $m \times n$ feature matrix, given in eq. (3):

$$\mathbf{X} = [\mathbf{x}_1 \cdots \mathbf{x}_n] \quad (3)$$

- $\mathbf{y}$ is the $m \times 1$ target vector
- Eq. (21) is called the *Normal Equation*.
- See the proof of the normal equation in Appendix (pages 70 to 72).

The Normal Equation: Code Example

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - ① cells 72 and 73
- See [/models/p2_shallow_learning:](#)
 - ① cell 1

Gradient Descent: The Motivation

- Unlike the normal equation which directly gives the closed-form expression of the optimal solution, *Gradient Descent* iteratively updates the parameters to approach the optimal solution.
- **Q:** Why do we need this iterative approach to estimate the optimal solution, when we already know its closed-form expression?

Gradient Descent: The Motivation

- Unlike the normal equation which directly gives the closed-form expression of the optimal solution, *Gradient Descent* iteratively updates the parameters to approach the optimal solution.
- **Q:** Why do we need this iterative approach to estimate the optimal solution, when we already know its closed-form expression?
- **A:** It is closely related to the good practice discussed on page 19 (see the explanation of the good practice in Appendix (pages 64 to 66)).



Good practice

- When m is very large (e.g., millions):
 - it is recommended to use SGDRegressor
- When m is relatively small (e.g., thousands):
 - when $m \gg n$:
 - it is recommended to use LinearRegression (which is faster)
 - when $m \ll n$:
 - it is recommended to use SGDRegressor (which is faster)

Batch / Stochastic / Mini-Batch Gradient Descent

- Based on the amount of training data we use in each epoch (i.e., iteration) of updating the parameters, we can divide gradient descent into three categories:
 - *Batch Gradient Descent* (BGD): use all the training data
 - *Stochastic Gradient Descent* (SGD): use only one training sample
 - *Mini-batch Gradient Descent* (MBGD): use a *Mini-Batch* of training samples (something between all the training data and only one training sample)
- We can think of BGD and SGD as two extremes:
 - BGD uses as many training data as possible
 - SGD uses as few training data as possible
- In that sense, MBGD is a trade-off between the two extremes.

BGD: The Idea

- The idea of BGD is iteratively updating the parameters in such a way that MSE gets minimized.
- This is realized by the following updating rule, which repeatedly updates the parameters $\boldsymbol{\theta}$ (where $\boldsymbol{\theta} = [b \ w_1 \ \dots \ w_n]^\top$):

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \mathbf{p}_k. \quad (23)$$

Here

- $\boldsymbol{\theta}_k$ are the parameters in epoch k (the current round)
- $\boldsymbol{\theta}_{k+1}$ are the parameters in epoch $k + 1$ (the next round)
- $\mathbf{p}_k$ is a direction in epoch k along which we search for $\boldsymbol{\theta}_{k+1}$ from $\boldsymbol{\theta}_k$
- η_k is the *Learning Rate* (a positive scalar) that determines the step size in epoch k (how far we move from $\boldsymbol{\theta}_k$ along $\mathbf{p}_k$ to search for $\boldsymbol{\theta}_{k+1}$)
- Once we know the learning rate (η_k) and direction ($\mathbf{p}_k$), we can use eq. (23) to go from $\boldsymbol{\theta}_k$ to $\boldsymbol{\theta}_{k+1}$.

BGD: Two Analogies

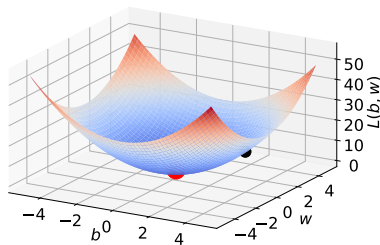


Figure 9: The surface plot of MSE of the linear regression in eq. (9).

$$\widehat{\text{SalePrice}} = b + w \times \text{LotFrontage}. \quad (9)$$

- Analogy 1: Based on fig. 9, we can think of MSE as a bowl-shaped valley.
- Analogy 2: Since the idea of BGD is iteratively updating the parameters in such a way that MSE gets minimized (e.g., from the black dot in fig. 9 to the red one), we can think of BGD as going down the bowl-shaped valley.

BGD: Direction and Learning Rate

- Based on the two analogies:
 - the direction matters because it determines where we go:
 - we could go uphill (i.e., increasing MSE) rather than downhill (decreasing MSE) if the direction were wrong
 - the learning rate matters because it determines the step size:
 - we could overshoot the bottom of the valley (i.e., the minimum of MSE) if the step size were too large
- It turns out that:
 - knowing the direction is easy
 - knowing the learning rate is difficult
- As a result, we will:
 - focus on how to find the direction for now
 - discuss how to find-tune the learning rate in:
 - [/p2_c2_s2_training_shallow_models](#)
 - [/p3_c2_s2_training_deep_neural_networks](#)

BGD: Finding the Direction

- The goal is to find the direction, $\mathbf{p}_k$, in the updating rule in eq. (23)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \mathbf{p}_k. \quad (23)$$

Here

- $\boldsymbol{\theta}_k$ are the parameters in epoch k (the current round)
- $\boldsymbol{\theta}_{k+1}$ are the parameters in epoch $k + 1$ (the next round)
- $\mathbf{p}_k$ is a direction in epoch k along which we search for $\boldsymbol{\theta}_{k+1}$ from $\boldsymbol{\theta}_k$
- η_k is the learning rate (a positive scalar) that determines the step size in epoch k (how far we move from $\boldsymbol{\theta}_k$ along $\mathbf{p}_k$ to search for $\boldsymbol{\theta}_{k+1}$)
- It turns out that $\mathbf{p}_k$ is the direction that leads to the steepest descent of MSE.
- It makes sense as, again, the idea of BGD is iteratively updating the parameters so that MSE gets minimized.

BGD: Finding the Direction

- The *Gradient*, $\mathbf{g}_k$, is the first-order derivative of MSE, $\mathcal{L}(\boldsymbol{\theta})$, with respect to the parameters in epoch k , $\boldsymbol{\theta}_k$:

$$\mathbf{g}_k = \nabla \mathcal{L}(\boldsymbol{\theta})^\top |_{\boldsymbol{\theta}=\boldsymbol{\theta}_k} . \quad (24)$$

- The gradient, $\mathbf{g}_k$, is also the direction that leads to the steepest ascent of MSE, $\mathcal{L}(\boldsymbol{\theta})$.
- Since the direction, $\mathbf{p}_k$, is the direction that leads to the steepest descent of MSE, then it is the opposite direction of gradient:

$$\mathbf{p}_k = -\mathbf{g}_k = -\nabla \mathcal{L}(\boldsymbol{\theta})^\top |_{\boldsymbol{\theta}=\boldsymbol{\theta}_k} . \quad (25)$$

- See the proof of eq. (25) in Appendix (pages 73 and 74).

BGD: The Math

- The updating rule of BGD was given in eq. (23)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \mathbf{p}_k. \quad (23)$$

- Based on eq. (25)

$$\mathbf{p}_k = -\mathbf{g}_k = -\nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k}, \quad (25)$$

we can write eq. (23) as

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \mathbf{g}_k = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k}. \quad (26)$$

- By deriving the gradient, $\nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k}$, we can write eq. (26) as

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \frac{2\eta_k}{m} \sum_{i=1}^m (y^i - \hat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top = \boldsymbol{\theta}_k + \frac{2\eta_k}{m} \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^\top (\mathbf{y} - \hat{\mathbf{y}}). \quad (27)$$

Here:

- m is the number of samples in the data
- $y^i / \hat{y}^i$ is the real / predicted target value of sample i , where

$$\hat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \begin{bmatrix} b & w_1 \cdots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta}_k \quad (28)$$

and $\mathbf{y} / \hat{\mathbf{y}}$ is the $m \times 1$ real / predicted target vector, where

$$\hat{\mathbf{y}} = b + w_1 \mathbf{x}_1 + \dots + w_n \mathbf{x}_n = \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix} \begin{bmatrix} b & w_1 \cdots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix} \boldsymbol{\theta}_k \quad (29)$$

- $\mathbf{x}^i$ is the $1 \times n$ feature vector of sample i , and $\mathbf{X}$ the $m \times n$ feature matrix
- See the proof of eq. (27) in Appendix (pages 75 and 76).

BGD: Code Example

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - ① cells 76 to 78
- See [/models/p2_shallow_learning:](#)
 - ① cell 2

BGD: The Loss

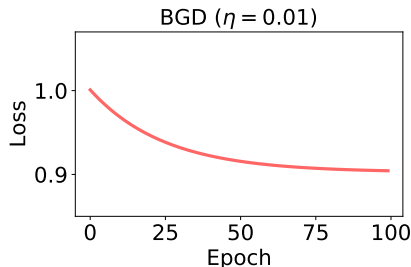


Figure 10: The training loss in BGD as a function of epoch.

- Fig. 10 shows that the training loss in BGD keeps decreasing when training proceeds.
- However, this does not mean the performance of the model keeps improving (we will discuss in [/p2_c2_s2_training_shallow_models](#)).

BGD: Pros and Cons

- Pros:
 - utilizes parallel computing to the most extent (making it efficient)
 - does not require shuffling the data (making it even more efficient)
 - relies on gradient calculated using all the training data (easier to converge to the optimal solution)
- Cons:
 - requires processing all the training data to complete one update of the parameters (not suitable for large datasets)



Takeaway

- BGD is suitable for small datasets.

SGD: The Idea

- Unlike BGD that requires processing all the training data to complete one update of the parameters, Stochastic Gradient Descent (SGD) only requires processing one training sample to complete one update.
- This is realized by the following updating rule, which, in each epoch k , updates the parameters $\boldsymbol{\theta}$ (where $\boldsymbol{\theta} = [b \ w_1 \dots w_n]^\top$) after processing each training sample i :

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i + \eta_k \mathbf{p}_k^i. \quad (30)$$

Here

- $\boldsymbol{\theta}_k^i$ are the parameters in epoch k after the update using sample i
- $\boldsymbol{\theta}_k^{i+1}$ are the parameters in epoch k after the update using sample $i + 1$
- $\mathbf{p}_k^i$ is a direction in epoch k along which we search for $\boldsymbol{\theta}_k^{i+1}$ from $\boldsymbol{\theta}_k^i$
- η_k is the learning rate that determines the step size in epoch k (how far we move from $\boldsymbol{\theta}_k^i$ along $\mathbf{p}_k^i$ to search for $\boldsymbol{\theta}_k^{i+1}$)

SGD: Squared Error

- Since SGD updates the parameters using only one training sample, the loss function is the *Squared Error*, $\mathcal{L}(\boldsymbol{\theta}^i)$, which measures the squared difference between the real target value and the predicted target value, with respect to sample i :

$$\mathcal{L}^i(\boldsymbol{\theta}) = (y^i - \hat{y}^i)^2. \quad (31)$$

Here:

- y^i is the real target value of sample i
- $\hat{y}^i$ is the predicted target value of sample i , given in eq. (28):

$$\hat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta} \quad (28)$$

SGD: Finding the Direction

- The goal is to find the direction, $\mathbf{p}_k^i$, in the updating rule in eq. (30)

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i + \eta_k \mathbf{p}_k^i. \quad (30)$$

where $\mathbf{p}_k^i$ is a direction in epoch k along which we search for $\boldsymbol{\theta}_k^{i+1}$ from $\boldsymbol{\theta}_k^i$.

- It turns out that $\mathbf{p}_k^i$ is the direction that leads to the steepest descent of SE, $\mathcal{L}(\boldsymbol{\theta}^i)$, given in eq. (31):

$$\mathcal{L}(\boldsymbol{\theta}^i) = (y^i - \hat{y}^i)^2. \quad (31)$$

- It this sense, we can also think of SGD as an approximation of BGD:
 - BGD aims to minimize MSE, $\mathcal{L}(\boldsymbol{\theta})$, by taking the direction of the steepest descent of MSE, $\mathcal{L}(\boldsymbol{\theta})$
 - SGD also aims to minimize MSE, $\mathcal{L}(\boldsymbol{\theta})$, but by taking the direction of the steepest descent of SE, $\mathcal{L}(\boldsymbol{\theta}^i)$

SGD: Finding the Direction

- The gradient, $\mathbf{g}_k^i$, is the first-order derivative of SE, $\mathcal{L}(\boldsymbol{\theta}^i)$, with respect to the parameters in epoch k , updated using sample i , $\boldsymbol{\theta}_k^i$:

$$\mathbf{g}_k^i = \nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^i}. \quad (32)$$

- The gradient, $\mathbf{g}_k^i$, is also the direction that leads to the steepest ascent of SE.
- Since the direction, $\mathbf{p}_k^i$, is the direction that leads to the steepest descent of SE, it is the opposite direction of gradient:

$$\mathbf{p}_k^i = -\mathbf{g}_k^i = -\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^i}. \quad (33)$$

- See the proof of eq. (33) in Appendix (pages 77 and 78).

SGD: The Math

- The updating rule of SGD was given in eq. (30)

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i + \eta_k \mathbf{p}_k^i. \quad (30)$$

- Based on eq. (33)

$$\mathbf{p}_k^i = -\mathbf{g}_k^i = -\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^i}, \quad (33)$$

we can write eq. (30) as

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i - \eta_k \mathbf{g}_k^i = \boldsymbol{\theta}_k^i - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^i}. \quad (34)$$

- By deriving the gradient, $\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^i}$, we can write eq. (34) as

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i + 2\eta_k (y^i - \widehat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top. \quad (35)$$

Here:

- y^i is the real target value of sample i
- $\widehat{y}^i$ is the predicted target value of sample i , given in eq. (28)

$$\widehat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \begin{bmatrix} b & w_1 \cdots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta}_k^i \quad (28)$$

- $\mathbf{x}^i$ is the $1 \times n$ feature vector of sample i
- See the proof of eq. (35) in Appendix (pages 79 and 80).

SGD: The Implementation

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - ① cells 79 to 81
- See [/models/p2_shallow_learning:](#)
 - ① cell 3

SGD: The Loss

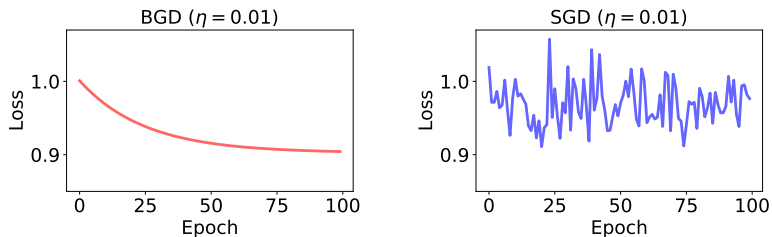


Figure 11: The training loss of BGD and SGD with learning rate 0.01.

- Fig. 11 shows that the loss of BGD is much smoother than the loss of SGD.
- This is not surprising since, as we mentioned earlier, SGD (taking the steepest descent of SE) is an approximation of BGD (taking the steepest descent of MSE).

SGD: Sklearn SGDRegressor

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - 1 cell 63

SGD: Pros and Cons

- Pros:

- allows processing only one training sample to complete one update of the parameters, making it more suitable for large datasets (compared to BGD)
- relies on gradient calculated using each training sample, so that the loss of SGD is bumpier than the loss of BGD, making SGD easier to escape local minimum (compared to BGD)

- Cons:

- utilizes parallel computing to the least extent, making it less efficient (than BGD)
- requires shuffling the data in the beginning of each epoch, making it less efficient (than BGD)
- relies on gradient calculated using each training sample, so that the loss of SGD is bumpier than the loss of BGD, making it more difficult to converge to the global minimum (compared to BGD)

BGD VS SGD

Takeaway

- BGD is suitable for small datasets.
- SGD is suitable for large datasets.

MBGD: The Idea

- Unlike BGD / SGD that uses all the training data / only one training sample to complete one update of the parameters, MBGD (Mini-Batch Gradient Descent) uses a *Mini-Batch* of training samples (something between all the training data and only one training sample) to do so.
- This is realized by the following updating rule, which, in each epoch k , updates the parameters $\boldsymbol{\theta}$ (where $\boldsymbol{\theta} = [b \ w_1 \ \dots \ w_n]^\top$) after processing each mini-batch:

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \mathbf{p}_k^j. \quad (36)$$

Here

- $\boldsymbol{\theta}_k^j$ are the parameters in epoch k after the update using mini-batch $\mathbf{mb}^j$
- $\boldsymbol{\theta}_k^{j+1}$ are the parameters in epoch k after the update using mini-batch $\mathbf{mb}^{j+1}$
- $\mathbf{p}_k^j$ is a direction in epoch k along which we search for $\boldsymbol{\theta}_k^{j+1}$ from $\boldsymbol{\theta}_k^j$
- η_k is the learning rate that determines the step size in epoch k (how far we move from $\boldsymbol{\theta}_k^j$ along $\mathbf{p}_k^j$ to search for $\boldsymbol{\theta}_k^{j+1}$)

MBGD: Mean Squared Error

- Since MBGD updates the parameters using a mini-batch of training samples, the loss function is MSE, $\mathcal{L}(\boldsymbol{\theta}^j)$, which measures the average squared difference between the real target value and the predicted target value, across the samples in mini-batch $\mathbf{mb}^j$:

$$\mathcal{L}(\boldsymbol{\theta}^j) = \frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i)^2. \quad (37)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- y^i is the real target value of sample i
- $\hat{y}^i$ is the predicted target value of sample i , given in eq. (28)

$$\hat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \begin{bmatrix} b & w_1 \cdots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta} \quad (28)$$

MBGD: Finding the Direction

- The goal is to find the direction, $\mathbf{p}_k^j$, in the updating rule in eq. (36)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \mathbf{p}_k^j, \quad (36)$$

where $\mathbf{p}_k^j$ is a direction in epoch k along which we search for $\boldsymbol{\theta}_k^{j+1}$ from $\boldsymbol{\theta}_k^j$.

- It turns out that $\mathbf{p}_k^j$ is the direction that leads to the steepest descent of MSE with respect to mini-batch $\mathbf{mb}^j$, $\mathcal{L}(\boldsymbol{\theta}^j)$.
- In this sense, MBGD is a better approximation of BGD (than SGD):
 - BGD aims to minimize MSE, $\mathcal{L}(\boldsymbol{\theta})$, by taking the direction of the steepest descent of MSE, $\mathcal{L}(\boldsymbol{\theta})$
 - SGD aims to minimize MSE, $\mathcal{L}(\boldsymbol{\theta})$, by taking the direction of the steepest descent of SE, $\mathcal{L}(\boldsymbol{\theta}^i)$
 - MBGD aims to minimize MSE, $\mathcal{L}(\boldsymbol{\theta})$, by taking the direction of the steepest descent of MSE with respect to mini-batch $\mathbf{mb}^j$, $\mathcal{L}(\boldsymbol{\theta}^j)$

MBGD: Finding the Direction

- The gradient, $\mathbf{g}_k^j$, is the first-order derivative of MSE, $\mathcal{L}(\boldsymbol{\theta}^j)$, with respect to the parameters in epoch k , updated using mini-batch $\mathbf{mb}^j$, $\boldsymbol{\theta}_k^j$:

$$\mathbf{g}_k^j = \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}. \quad (38)$$

- The gradient, $\mathbf{g}_k^j$, is also the direction that leads to the steepest ascent of $\mathcal{L}(\boldsymbol{\theta}^j)$.
- Since the direction, $\mathbf{p}_k^j$, is the direction that leads to the steepest descent of $\mathcal{L}(\boldsymbol{\theta}^j)$, it is the opposite direction of gradient:

$$\mathbf{p}_k^j = -\mathbf{g}_k^j = -\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}. \quad (39)$$

- See the proof of eq. (39) in Appendix (pages 81 and 82).

MBGD: The Math

- The updating rule of MBGD was given in eq. (36)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \mathbf{p}_k^j. \quad (36)$$

- Based on eq. (39)

$$\mathbf{p}_k^j = -\mathbf{g}_k^j = -\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}, \quad (39)$$

we can write eq. (36) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j - \eta_k \mathbf{g}_k^j = \boldsymbol{\theta}_k - \eta_k \nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}. \quad (40)$$

- By deriving the gradient, $\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top \big|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j}$, we can write eq. (40) as

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{2\eta_k}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \widehat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top = \boldsymbol{\theta}_k^j + \frac{2\eta_k}{|\mathbf{mb}^j|} \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix}^\top (\mathbf{y}^j - \widehat{\mathbf{y}}^j). \quad (41)$$

Here:

- $|\mathbf{mb}^j|$ is the number of samples in mini-batch $\mathbf{mb}^j$
- $y^i / \widehat{y}^i$ is the real / predicted target value of sample i , given in eq. (28)

$$\widehat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix} \boldsymbol{\theta}_k^j \quad (28)$$

and $\mathbf{y}^j / \widehat{\mathbf{y}}^j$ is the $|\mathbf{mb}^j| \times 1$ real / predicted target vector of $\mathbf{mb}^j$, where

$$\widehat{\mathbf{y}}^j = \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix} \begin{bmatrix} b & w_1 \dots w_n \end{bmatrix}^\top = \begin{bmatrix} 1 & \mathbf{X}^j \end{bmatrix} \boldsymbol{\theta}_k^j \quad (42)$$

- $\mathbf{x}^i$ is the $1 \times n$ feature vector of sample i , and $\mathbf{X}^j$ the $|\mathbf{mb}^j| \times n$ feature matrix of $\mathbf{mb}^j$
- See the proof of eq. (41) in Appendix (pages 83 and 84).

MBGD: The Implementation

- See [/p2_c2_s1_linear_regression/code_example:](#)
 - ① cells 82 to 84
- See [/models/p2_shallow_learning:](#)
 - ① cell 4



Good practice

- It is recommended to use small batches (from 2 to 32) in MBGD [Masters and Luschi, 2018].

MBGD: The Loss

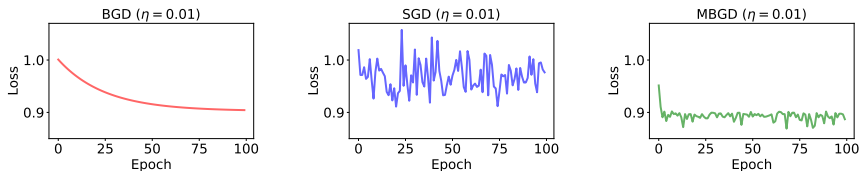


Figure 12: The training loss of BGD, SGD and MBGD with learning rate 0.01.

- Fig. 12 shows that:
 - the loss of MBGD is bumpier than the loss of BGD
 - the loss of MBGD is smoother than the loss of SGD
- This is not surprising since, as we mentioned earlier:
 - BGD takes the steepest descent of MSE
 - SGD takes the steepest descent of SE
 - MBGD takes the steepest descent of mini-batch MSE
- Compared to SGD, MBGD is a better approximation of BGD.

MBGD: Pros and Cons

• Pros:

- allows processing only a mini-batch of training samples to complete one update of the parameters, making it more suitable for large datasets (compared to BGD)
- relies on gradient calculated using a mini-batch of training samples, making it easier to:
 - escape local minimum (compared to BGD)
 - converge to the global minimum (compared to SGD)
- utilizes parallel computing to some extent, making it more efficient (than SGD)

• Cons:

- utilizes parallel computing to some extent, making it less efficient (than BGD)
- requires shuffling the data in the beginning of each epoch, making it less efficient (than BGD)
- relies on gradient calculated using a mini-batch of training samples, making it more difficult to:
 - converge to the global minimum (compared to BGD)
 - escape local minimum (compared to SGD)

BGD VS SGD VS MBGD

Takeaway

- BGD is suitable for small datasets.
 - Both SGD and MBGD are suitable for large datasets.
 - MBGD is more popular in deep learning.
-
- Hereafter, we will use MBGD as the default for gradient descent.
 - This is mainly because:
 - in theory, MBGD reduces to BGD / SGD when the mini-batch contains all the data / only one sample, so that we can slightly tweak the equations for MBGD (with respect to the mini-batch size) to get the equations for BGD and SGD
 - in practice, MBGD is more popular in deep learning

Time Complexity: Page 19

- In simple words, the time complexity of an algorithm is the theoretical run time of the algorithm.
- Since the run time increases when the size of the input increases (e.g., counting to 100 takes more time than counting to 10), we would like to know the *Asymptotic Order of Growth* of the time, which measures a bound of the run time, when the input is large.
- One such bound is the upper bound, denoted by big O (see a rigorous discussion in Chapter 3 of [Cormen et al., 2009]).
- For example, when we say the time complexity of an algorithm is, say $O(n^3)$ (where n is the size of the input), what we mean is that the theoretical run time is lower than n^3 (as $O(n^3)$ is an upper bound), when n is large.

LinearRegression VS SGDRegressor: Page 19

- With m being the number of training samples, n the number of features and k the number of epochs:

- the time complexity of LinearRegression (which solves the normal equation) is

$$\max(O(n^2m), O(n^3)) = \begin{cases} O(n^2m), & \text{if } m > n, \\ O(n^3), & \text{otherwise,} \end{cases} \quad (43)$$

- if we only want to update the parameters c times, the time complexity of doing so is still the same as those in eq. (43)
- this is because we still have to solve the normal equation, with complexity in eq. (43)
- the time complexity of SGDRegressor (which uses SGD) updating the parameters km times is

$$O(kmn), \quad (44)$$

- if we only want to update the parameters c times, the time complexity of doing so is

$$O(cn). \quad (45)$$

- See the proof of eqs. (43), (44) and (45) in Appendix (pages 67 and 68).

LinearRegression VS SGDRegressor: Page 19

- With the time complexities in eqs. (43), (44) and (45) , we can explain (see the text in red) the good practice on page 19.



Good practice

- When m is very large (e.g., millions):
 - it is recommended to use SGDRegressor
 - because we can use SGDRegressor to update the parameters c times to have a reasonably good estimation of the optimal solutions, whose complexity (eq. (45)) could be much lower than that of LinearRegression (eq. (43))
- When m is relatively small (e.g., thousands):
 - when $m \gg n$:
 - it is recommended to use LinearRegression (which is faster)
 - because the complexity of LinearRegression (eq. (43)) could be lower than that of SGDRegressor (eq. (44)), when $k > n$
 - when $m \ll n$:
 - it is recommended to use SGDRegressor (which is faster)
 - because the complexity of SGDRegressor (eq. (44)) could be lower than that of LinearRegression (eq. (43)), when $km < n^2$

Proof of Time Complexity: Page 65

- To solve the normal equation in eq. (21)

$$\boldsymbol{\theta}^* = ([\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}])^{-1} [\mathbf{1} \quad \mathbf{X}]^\top \mathbf{y}, \quad (21)$$

we need the following steps (from left to right):

- ① calculating $[\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}]$, with $O(n^2m)$ time complexity
 - ② calculating $([\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}])^{-1}$, with typically $O(n^3)$ time complexity
 - ③ calculating $([\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}])^{-1} [\mathbf{1} \quad \mathbf{X}]^\top$, with $O(n^2m)$ time complexity
 - ④ calculating $([\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}])^{-1} [\mathbf{1} \quad \mathbf{X}]^\top \mathbf{y}$, with $O(nm)$ time complexity
- According to the *Additivity Property* of the big O notation (which says the overall time complexity of an algorithm is the largest one across the individual steps of the algorithm), the time complexity of solving the normal equation is

$$\max(O(n^2m), O(n^3)), \quad (46)$$

which proves the claim in eq. (43) on page 65. □

Proof of Time Complexity: Page 65

- SGD uses the updating rule in eq. (35) to update the parameters once:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + 2\eta_k (y^i - \widehat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top. \quad (35)$$

- The time complexity of calculating $2\eta_k (y^i - \widehat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top$ in eq. (35) is

$$O(n). \quad (47)$$

- If we update the parameters c times, the time complexity is

$$O(cn), \quad (48)$$

which proves the claim in eq. (45).

- If there are k epochs and m training samples, then we update the parameters km times, so that the time complexity is

$$O(kmn), \quad (49)$$

which proves the claim in eq. (44). □

Proof of First-Order Condition: Page 31

- Eq. (19) shows the first-order approximation of $\mathcal{L}(\boldsymbol{\theta})$ about $\boldsymbol{\theta}^*$

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^*) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*). \quad (19)$$

- Let $\boldsymbol{\theta} = \boldsymbol{\theta}^* + \Delta\boldsymbol{\theta}$ (where $\Delta\boldsymbol{\theta} > 0$), then based on eq. (19) we have

$$\mathcal{L}(\boldsymbol{\theta}^* + \Delta\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^*) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} \Delta\boldsymbol{\theta}. \quad (50)$$

- Let $\boldsymbol{\theta} = \boldsymbol{\theta}^* - \Delta\boldsymbol{\theta}$ (where $\Delta\boldsymbol{\theta} > 0$), then based on eq. (19) we have

$$\mathcal{L}(\boldsymbol{\theta}^* - \Delta\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^*) - \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} \Delta\boldsymbol{\theta}. \quad (51)$$

- Since $\boldsymbol{\theta}^*$ is the optimal solution (that minimizes $\mathcal{L}(\boldsymbol{\theta})$), we have

$$\mathcal{L}(\boldsymbol{\theta}^* + \Delta\boldsymbol{\theta}) - \mathcal{L}(\boldsymbol{\theta}^*) \geq 0 \leq \mathcal{L}(\boldsymbol{\theta}^* - \Delta\boldsymbol{\theta}) - \mathcal{L}(\boldsymbol{\theta}^*). \quad (52)$$

- Based on eqs. (50) to (52) we have

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} \geq 0 \geq \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*}, \quad (53)$$

which proves the first-order condition in eq. (20) on page 31. □

Proof of the Normal Equation: Page 32

- Eq. (11) shows the loss function (MSE) of linear regression:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (y^i - \hat{y}^i)^2 \quad \text{where} \quad \hat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i. \quad (11)$$

- We can write eq. (11) in matrix form:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} (\mathbf{y} - \hat{\mathbf{y}})^\top (\mathbf{y} - \hat{\mathbf{y}}). \quad (54)$$

- Eq. (20) shows the first-order condition:

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top |_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0. \quad (20)$$

- By substituting eq. (54) into eq. (20) we have

$$\nabla \mathcal{L}(\boldsymbol{\theta}) |_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = \nabla \frac{1}{m} (\mathbf{y} - \hat{\mathbf{y}})^\top (\mathbf{y} - \hat{\mathbf{y}}) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0. \quad (55)$$

Proof of the Normal Equation: Page 32

- Since eq. (55) still holds after removing $\frac{1}{m}$, then

$$\nabla(\mathbf{y} - \hat{\mathbf{y}})^\top (\mathbf{y} - \hat{\mathbf{y}})|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0. \quad (56)$$

Here $\hat{\mathbf{y}}$ is the predicted target vector, given in eq. (2):

$$\hat{\mathbf{y}} = [\mathbf{1} \quad \mathbf{X}] \boldsymbol{\theta}. \quad (2)$$

- By substituting eq. (2) into eq. (56) we have

$$\nabla (\mathbf{y} - [\mathbf{1} \quad \mathbf{X}] \boldsymbol{\theta})^\top (\mathbf{y} - [\mathbf{1} \quad \mathbf{X}] \boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0, \quad (57)$$

which can be written as

$$\nabla \left(\mathbf{y}^\top \mathbf{y} - 2\boldsymbol{\theta}^\top [\mathbf{1} \quad \mathbf{X}]^\top \mathbf{y} + \boldsymbol{\theta}^\top [\mathbf{1} \quad \mathbf{X}]^\top [\mathbf{1} \quad \mathbf{X}] \boldsymbol{\theta} \right) \Big|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} = 0. \quad (58)$$

Proof of the Normal Equation: Page 32

- By calculating the derivation in eq. (58) we have

$$-2 \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \mathbf{y} + 2 \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix} \boldsymbol{\theta}^* = 0, \quad (59)$$

which can be written as

$$\begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix} \boldsymbol{\theta}^* = \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \mathbf{y}. \quad (60)$$

- By multiplying $(\begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix})^{-1}$ on both sides of eq. (60) we have

$$\boldsymbol{\theta}^* = (\begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix})^{-1} \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^T \mathbf{y}, \quad (61)$$

which proves the normal equation in eq. (21) on page 32. □

Proof of the Search Direction: Page 40

- The first-order approximation of MSE, $\mathcal{L}(\boldsymbol{\theta})$, about the optimal solution (which minimizes $\mathcal{L}(\boldsymbol{\theta})$), $\boldsymbol{\theta}^*$, is given in eq. (19)

$$\mathcal{L}(\boldsymbol{\theta}) \approx \mathcal{L}(\boldsymbol{\theta}^*) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}^*} (\boldsymbol{\theta} - \boldsymbol{\theta}^*). \quad (19)$$

- Similar to eq. (19), the first-order approximation of MSE in epoch $k + 1$, $\mathcal{L}(\boldsymbol{\theta}_{k+1})$, about the parameters in epoch k , $\boldsymbol{\theta}_k$, is

$$\mathcal{L}(\boldsymbol{\theta}_{k+1}) \approx \mathcal{L}(\boldsymbol{\theta}_k) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k} (\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k) = \mathcal{L}(\boldsymbol{\theta}_k) + \mathbf{g}_k(\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k). \quad (62)$$

- In BGD, the parameters are updated iteratively using the updating rule in eq. (23)

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \eta_k \mathbf{p}_k, \quad (23)$$

which can be written as

$$\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k = \eta_k \mathbf{p}_k. \quad (63)$$

- By substituting eq. (63) into eq. (62) we have

$$\mathcal{L}(\boldsymbol{\theta}_{k+1}) \approx \mathcal{L}(\boldsymbol{\theta}_k) + \eta_k \mathbf{g}_k \mathbf{p}_k, \quad (64)$$

which can be written as

$$\mathcal{L}(\boldsymbol{\theta}_{k+1}) - \mathcal{L}(\boldsymbol{\theta}_k) \approx \eta_k \mathbf{g}_k \mathbf{p}_k. \quad (65)$$

Proof of the Search Direction: Page 40

- Since $\mathbf{g}_k \mathbf{p}_k$ in eq. (65) is the dot product between $\mathbf{g}_k$ and $\mathbf{p}_k$, we can write eq. (65) as

$$\mathcal{L}(\boldsymbol{\theta}_{k+1}) - \mathcal{L}(\boldsymbol{\theta}_k) \approx \eta_k \|\mathbf{g}_k\| \|\mathbf{p}_k\| \cos(\alpha). \quad (66)$$

Here

- η_k is the learning rate in epoch k
- $\|\mathbf{g}_k\|$ and $\|\mathbf{p}_k\|$ are the magnitude of $\mathbf{g}_k$ and $\mathbf{p}_k$
- α is the angle between $\mathbf{g}_k$ and $\mathbf{p}_k$
- Since the goal of BGD is finding the optimal solution, $\boldsymbol{\theta}^*$, which minimizes MSE, we want $\boldsymbol{\theta}_{k+1}$ to be as close to $\boldsymbol{\theta}^*$ as possible.
- In other words, we want to minimize the difference in eq. (66), $\mathcal{L}(\boldsymbol{\theta}_{k+1}) - \mathcal{L}(\boldsymbol{\theta}_k)$.
- Since the difference is the minimum when $\alpha = 180^\circ$:

$$\arg \min_{\alpha} \eta_k \|\mathbf{g}_k\| \|\mathbf{p}_k\| \cos(\alpha) = 180^\circ, \quad (67)$$

$\mathbf{p}_k$ takes the opposite direction of $\mathbf{g}_k$.

- Assume $\mathbf{p}_k$ and $\mathbf{g}_k$ have the same magnitude (which is fine due to η_k), we have

$$\mathbf{p}_k = -\mathbf{g}_k, \quad (68)$$

which proves the claim in eq. (25) on page 40. □

Proof of the Updating Rule: Page 41

- We can write the gradient (i.e., first-order derivative) of MSE, $\nabla \mathcal{L}(\boldsymbol{\theta})^\top$, as

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top = \left[\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}) \quad \frac{\partial}{\partial w_1} \mathcal{L}(\boldsymbol{\theta}) \cdots \frac{\partial}{\partial w_n} \mathcal{L}(\boldsymbol{\theta}) \right]^\top, \quad (69)$$

where MSE, $\mathcal{L}(\boldsymbol{\theta})$, is given in eq. (11)

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (y^i - \widehat{y}^i)^2 \quad \text{where} \quad \widehat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i. \quad (11)$$

- Based on eq. (11), we can write $\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta})$ as

$$\begin{aligned} \frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}) &= \frac{\partial}{\partial b} \left(\frac{1}{m} \sum_{i=1}^m (y^i - \widehat{y}^i)^2 \right), \\ &= \frac{2}{m} \sum_{i=1}^m (y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial b} (y^i - \widehat{y}^i), \\ &= \frac{2}{m} \sum_{i=1}^m (y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial b} (y^i - (b + w_1 x_1^i + \dots + w_n x_n^i)), \\ &= \frac{2}{m} \sum_{i=1}^m (y^i - \widehat{y}^i) \cdot (-1). \end{aligned} \quad (70)$$

Proof of the Updating Rule: Page 41

- Based on eq. (11), we can write $\frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta})$ as

$$\begin{aligned} \frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}) &= \frac{\partial}{\partial w_j} \left(\frac{1}{m} \sum_{i=1}^m (y^i - \hat{y}^i)^2 \right), \\ &= \frac{2}{m} \sum_{i=1}^m (y^i - \hat{y}^i) \cdot \frac{\partial}{\partial w_j} (y^i - (b + w_1 x_1^i + \dots + w_n x_n^i)), \\ &= \frac{2}{m} \sum_{i=1}^m (y^i - \hat{y}^i) \cdot (-x_j^i). \end{aligned} \quad (71)$$

- By substituting eqs. (70) and (71) into eq. (69), we have

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top = -\frac{2}{m} \sum_{i=1}^m (y^i - \hat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top = -\frac{2}{m} \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^\top (\mathbf{y} - \hat{\mathbf{y}}). \quad (72)$$

- By substituting eq. (72) into eq. (26), we have

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \frac{2\eta_k}{m} \sum_{i=1}^m (y^i - \hat{y}^i) \begin{bmatrix} 1 & \mathbf{x}^i \end{bmatrix}^\top = \boldsymbol{\theta}_k + \frac{2\eta_k}{m} \begin{bmatrix} 1 & \mathbf{X} \end{bmatrix}^\top (\mathbf{y} - \hat{\mathbf{y}}), \quad (73)$$

which proves the claim in eq. (27) on page 41. □

Proof of the Search Direction: Page 48

- Similar to eq. (19), the first-order approximation of SE in epoch k after the update using sample $i + 1$, $\mathcal{L}(\boldsymbol{\theta}_k^{i+1})$, about the parameters in epoch k after the update using sample i , $\boldsymbol{\theta}_k^i$, is

$$\mathcal{L}(\boldsymbol{\theta}_k^{i+1}) \approx \mathcal{L}(\boldsymbol{\theta}_k^i) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^i} (\boldsymbol{\theta}_k^{i+1} - \boldsymbol{\theta}_k^i) = \mathcal{L}(\boldsymbol{\theta}_k^i) + \mathbf{g}_k^i (\boldsymbol{\theta}_k^{i+1} - \boldsymbol{\theta}_k^i). \quad (74)$$

- In SGD, the parameters are updated iteratively using the updating rule in eq. (30)

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i + \eta_k \mathbf{p}_k^i. \quad (30)$$

which can be written as

$$\boldsymbol{\theta}_k^{i+1} - \boldsymbol{\theta}_k^i = \eta_k \mathbf{p}_k^i. \quad (75)$$

- By substituting eq. (75) into eq. (74) we have

$$\mathcal{L}(\boldsymbol{\theta}_k^{i+1}) \approx \mathcal{L}(\boldsymbol{\theta}_k^i) + \eta_k \mathbf{g}_k^i \mathbf{p}_k^i, \quad (76)$$

which can be written as

$$\mathcal{L}(\boldsymbol{\theta}_k^{i+1}) - \mathcal{L}(\boldsymbol{\theta}_k^i) \approx \eta_k \mathbf{g}_k^i \mathbf{p}_k^i. \quad (77)$$

Proof of the Search Direction: Page 48

- Since $\mathbf{g}_k^i \mathbf{p}_k^i$ in eq. (77) is the dot product between $\mathbf{g}_k^i$ and $\mathbf{p}_k^i$, we can write eq. (77) as

$$\mathcal{L}(\boldsymbol{\theta}_k^{i+1}) - \mathcal{L}(\boldsymbol{\theta}_k^i) \approx \eta_k \|\mathbf{g}_k^i\| \|\mathbf{p}_k^i\| \cos(\alpha). \quad (78)$$

Here

- η_k is the learning rate in epoch k
- $\|\mathbf{g}_k^i\|$ and $\|\mathbf{p}_k^i\|$ are the magnitude of $\mathbf{g}_k^i$ and $\mathbf{p}_k^i$
- α is the angle between $\mathbf{g}_k^i$ and $\mathbf{p}_k^i$
- Since the goal of SGD is finding the optimal solution, $\boldsymbol{\theta}^*$, which minimizes MSE, we want $\boldsymbol{\theta}_k^i$ to be as close to $\boldsymbol{\theta}^*$ as possible.
- In other words, we want to minimize the difference in eq. (78), $\mathcal{L}(\boldsymbol{\theta}_k^{i+1}) - \mathcal{L}(\boldsymbol{\theta}_k^i)$.
- Since the difference is the minimum when $\alpha = 180^\circ$:

$$\arg \min_{\alpha} \eta_k \|\mathbf{g}_k^i\| \|\mathbf{p}_k^i\| \cos(\alpha) = 180^\circ, \quad (79)$$

$\mathbf{p}_k^i$ takes the opposite direction of $\mathbf{g}_k^i$.

- Assume $\mathbf{p}_k^i$ and $\mathbf{g}_k^i$ have the same magnitude (which is fine due to η_k), we have

$$\mathbf{p}_k^i = -\mathbf{g}_k^i, \quad (80)$$

which proves the claim in eq. (33) on page 48. □

Proof of the Updating Rule: Page 49

- We can write the gradient (i.e., first-order derivative) of SE, $\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top$, as

$$\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top = \left[\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^i) \quad \frac{\partial}{\partial w_1} \mathcal{L}(\boldsymbol{\theta}^i) \cdots \frac{\partial}{\partial w_n} \mathcal{L}(\boldsymbol{\theta}^i) \right]^\top, \quad (81)$$

where SE, $\mathcal{L}(\boldsymbol{\theta}^i)$, is given in eq. (31)

$$\mathcal{L}^i(\boldsymbol{\theta}) = (y^i - \widehat{y}^i)^2 \quad \text{where} \quad \widehat{y}^i = b + w_1 x_1^i + \dots + w_n x_n^i. \quad (31)$$

- Based on eq. (31), we can write $\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^i)$ as

$$\begin{aligned} \frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^i) &= \frac{\partial}{\partial b} (y^i - \widehat{y}^i)^2, \\ &= 2(y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial b} (y^i - \widehat{y}^i), \\ &= 2(y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial b} (y^i - (b + w_1 x_1^i + \dots + w_n x_n^i)), \\ &= 2(y^i - \widehat{y}^i) \cdot (-1). \end{aligned} \quad (82)$$

Proof of the Updating Rule: Page 49

- Based on eq. (31), we can write $\frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^i)$ as

$$\begin{aligned} \frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^i) &= \frac{\partial}{\partial w_j} (y^i - \widehat{y}^i)^2, \\ &= 2(y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial w_j} (y^i - (b + w_1 x_1^i + \dots + w_n x_n^i)), \\ &= 2(y^i - \widehat{y}^i) \cdot (-x_j^i). \end{aligned} \quad (83)$$

- By substituting eqs. (82) and (83) into eq. (81), we have

$$\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top = -2(y^i - \widehat{y}^i) [1 \quad \mathbf{x}^i]^\top. \quad (84)$$

- By substituting eq. (84) into eq. (34), we have

$$\boldsymbol{\theta}_k^{i+1} = \boldsymbol{\theta}_k^i + 2\eta_k (y^i - \widehat{y}^i) [1 \quad \mathbf{x}^i]^\top, \quad (85)$$

which proves the claim in eq. (35) on page 49. □

Proof of the Search Direction: Page 58

- Similar to eq. (19), the first-order approximation of MSE in epoch k after the update using mini-batch $\mathbf{mb}^{j+1}$, $\mathcal{L}(\boldsymbol{\theta}_k^{j+1})$, about the parameters in epoch k after the update using mini-batch $\mathbf{mb}^j$, $\boldsymbol{\theta}_k^j$, is

$$\mathcal{L}(\boldsymbol{\theta}_k^{j+1}) \approx \mathcal{L}(\boldsymbol{\theta}_k^j) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top|_{\boldsymbol{\theta}=\boldsymbol{\theta}_k^j} (\boldsymbol{\theta}_k^{j+1} - \boldsymbol{\theta}_k^j) = \mathcal{L}(\boldsymbol{\theta}_k^j) + \mathbf{g}_k^j (\boldsymbol{\theta}_k^{j+1} - \boldsymbol{\theta}_k^j). \quad (86)$$

- In MBGD, the parameters are updated iteratively using the updating rule in eq. (36)

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \eta_k \mathbf{p}_k^j. \quad (36)$$

which can be written as

$$\boldsymbol{\theta}_k^{j+1} - \boldsymbol{\theta}_k^j = \eta_k \mathbf{p}_k^j. \quad (87)$$

- By substituting eq. (87) into eq. (86) we have

$$\mathcal{L}(\boldsymbol{\theta}_k^{j+1}) \approx \mathcal{L}(\boldsymbol{\theta}_k^j) + \eta_k \mathbf{g}_k^j \mathbf{p}_k^j, \quad (88)$$

which can be written as

$$\mathcal{L}(\boldsymbol{\theta}_k^{j+1}) - \mathcal{L}(\boldsymbol{\theta}_k^j) \approx \eta_k \mathbf{g}_k^j \mathbf{p}_k^j. \quad (89)$$

Proof of the Search Direction: Page 58

- Since $\mathbf{g}_k^j \mathbf{p}_k^j$ in eq. (89) is the dot product between $\mathbf{g}_k^j$ and $\mathbf{p}_k^j$, we can write eq. (89) as

$$\mathcal{L}(\boldsymbol{\theta}_k^{j+1}) - \mathcal{L}(\boldsymbol{\theta}_k^j) \approx \eta_k \|\mathbf{g}_k^j\| \|\mathbf{p}_k^j\| \cos(\alpha). \quad (90)$$

Here

- η_k is the learning rate in epoch k
- $\|\mathbf{g}_k^j\|$ and $\|\mathbf{p}_k^j\|$ are the magnitude of $\mathbf{g}_k^j$ and $\mathbf{p}_k^j$
- α is the angle between $\mathbf{g}_k^j$ and $\mathbf{p}_k^j$
- Since the goal of MBGD is finding the optimal solution, $\boldsymbol{\theta}^*$, which minimizes MSE, we want $\boldsymbol{\theta}_k^j$ to be as close to $\boldsymbol{\theta}^*$ as possible.
- In other words, we want to minimize the difference in eq. (90), $\mathcal{L}(\boldsymbol{\theta}_k^{j+1}) - \mathcal{L}(\boldsymbol{\theta}_k^j)$.
- Since the difference is the minimum when $\alpha = 180^\circ$:

$$\arg \min_{\alpha} \eta_k \|\mathbf{g}_k^j\| \|\mathbf{p}_k^j\| \cos(\alpha) = 180^\circ, \quad (91)$$

$\mathbf{p}_k^j$ takes the opposite direction of $\mathbf{g}_k^j$.

- Assume $\mathbf{p}_k^j$ and $\mathbf{g}_k^j$ have the same magnitude (which is fine due to η_k), we have

$$\mathbf{p}_k^j = -\mathbf{g}_k^j, \quad (92)$$

which proves the claim in eq. (39) on page 58. □

Proof of the Updating Rule: Page 59

- We can write the gradient (i.e., first-order derivative) of MSE with respect to mini-batch $\mathbf{mb}^j$, $\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top$, as

$$\nabla \mathcal{L}(\boldsymbol{\theta}^j)^\top = \left[\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j) \quad \frac{\partial}{\partial w_1} \mathcal{L}(\boldsymbol{\theta}^j) \cdots \frac{\partial}{\partial w_n} \mathcal{L}(\boldsymbol{\theta}^j) \right]^\top, \quad (93)$$

where MSE, $\mathcal{L}(\boldsymbol{\theta}^j)$, is given in eq. (37)

$$\mathcal{L}(\boldsymbol{\theta}^j) = \frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \widehat{y}^i)^2. \quad (37)$$

- Based on eq. (37), we can write $\frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j)$ as

$$\begin{aligned} \frac{\partial}{\partial b} \mathcal{L}(\boldsymbol{\theta}^j) &= \frac{\partial}{\partial b} \left(\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \widehat{y}^i)^2 \right), \\ &= \frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial b} (y^i - \widehat{y}^i), \\ &= \frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \widehat{y}^i) \cdot \frac{\partial}{\partial b} (y^i - (b + w_1 x_1^i + \dots + w_n x_n^i)), \\ &= \frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \widehat{y}^i) \cdot (-1). \end{aligned} \quad (94)$$

Proof of the Updating Rule: Page 59

- Based on eq. (37), we can write $\frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^i)$ as

$$\begin{aligned}
 \frac{\partial}{\partial w_j} \mathcal{L}(\boldsymbol{\theta}^j) &= \frac{\partial}{\partial w_j} \left(\frac{1}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i)^2 \right), \\
 &= \frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i) \cdot \frac{\partial}{\partial w_j} (y^i - \hat{y}^i), \\
 &= \frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i) \cdot \frac{\partial}{\partial w_j} (y^i - (b + w_1 x_1^i + \dots + w_n x_n^i)), \\
 &= \frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i) \cdot (-x_j^i).
 \end{aligned} \tag{95}$$

- By substituting eqs. (94) and (95) into eq. (93), we have

$$\nabla \mathcal{L}(\boldsymbol{\theta}^i)^\top = -\frac{2}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i) [1 \quad \mathbf{x}^i]^\top = -\frac{2}{|\mathbf{mb}^j|} [1 \quad \mathbf{X}^j]^\top (\mathbf{y}^j - \hat{\mathbf{y}}^j). \tag{96}$$

- By substituting eq. (96) into eq. (40), we have

$$\boldsymbol{\theta}_k^{j+1} = \boldsymbol{\theta}_k^j + \frac{2\eta_k}{|\mathbf{mb}^j|} \sum_{i \in \mathbf{mb}^j} (y^i - \hat{y}^i) [1 \quad \mathbf{x}^i]^\top = \boldsymbol{\theta}_k^j + \frac{2\eta_k}{|\mathbf{mb}^j|} [1 \quad \mathbf{X}^j]^\top (\mathbf{y}^j - \hat{\mathbf{y}}^j), \tag{97}$$

which proves the claim in eq. (41) on page 59. □

Bibliography I



Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009).

Introduction to Algorithms.

MIT press.



Masters, D. and Luschi, C. (2018).

Revisiting Small Batch Training for Deep Neural Networks.

arXiv preprint arXiv:1804.07612.